



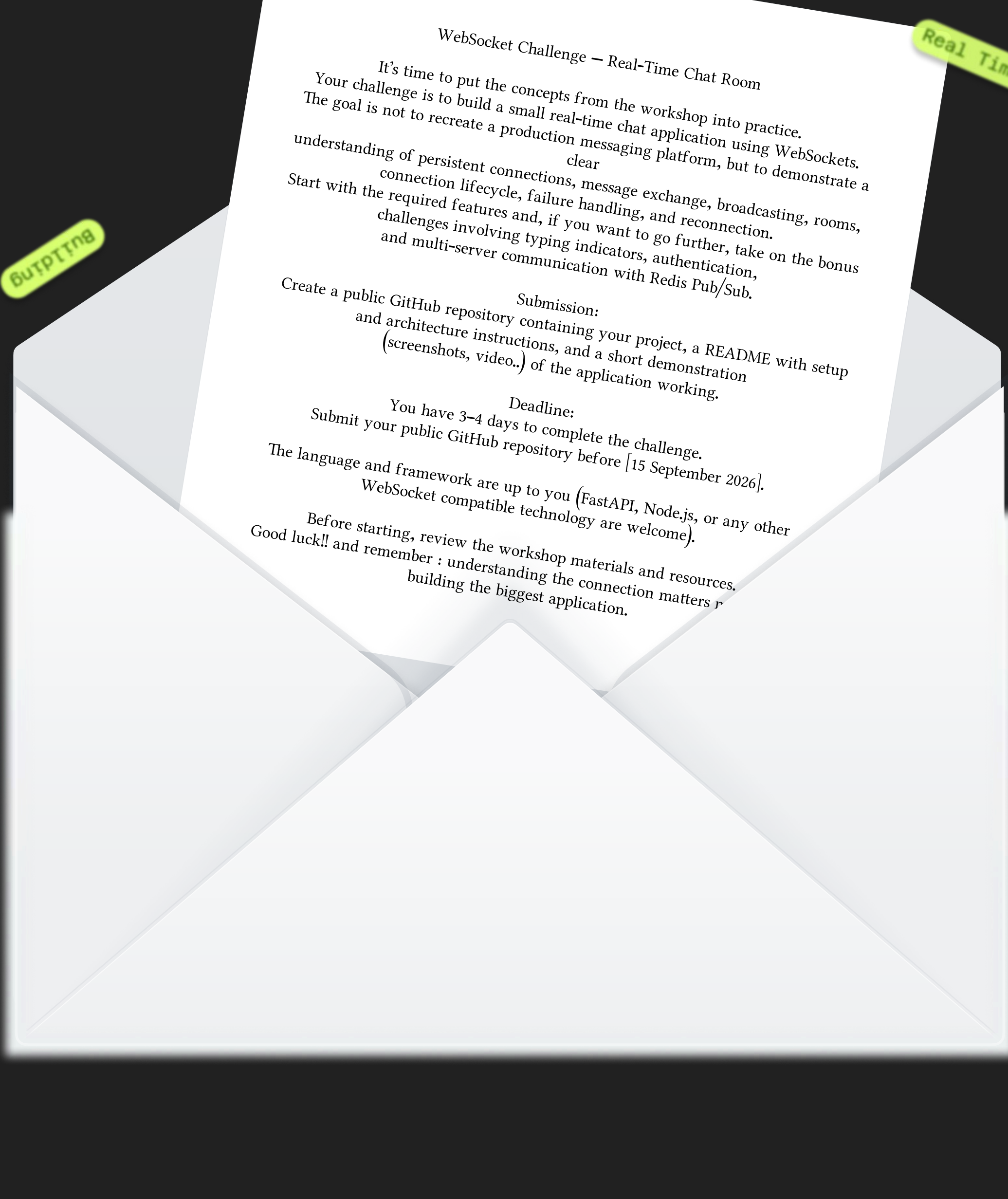
WebSocket Challenge

Real-time

Chat-room

Build a Real-Time App

WebSocket Challenge is here!!



Objective

Build a small real-time chat application using WebSockets.

The goal is to practice the concepts covered during the workshop:

- WebSocket connection
- Connection lifecycle
- Sending and receiving messages
- Broadcasting
- Rooms
- Message protocols
- Connection status
- Disconnection handling

Requirements

1. WebSocket Connection

Create a WebSocket endpoint and allow clients to connect to the server.

The client should display its connection state:

Connected
Connecting...
Disconnected

2. User Id

When connecting, each client provides a username.

Example: Alice connected

3. Message Protocol

Messages must follow a defined structure.

For example:

```
{
  "type": "chat.message",
  "data": {
    "message": "Hello everyone!"
  }
}
```

4. Broadcasting

When a user sends a message, broadcast it to the other users in the same room.

The sender should also see their own message.

5. Rooms

Users must be able to join a room.

6. Join / Leave Events

When someone joins a room:

"Alice joined the room."

When they disconnect:

"Alice left the room."

7. Connection Handling

The application must correctly handle:

- New connections
- Normal disconnections
- Unexpected disconnections
- Invalid messages
- Empty messages

The server should not crash when a client disconnects unexpectedly.

8. Reconnection

If the connection is lost, the client should attempt to reconnect.

Use a simple delay strategy:

1s → 2s → 4s → 8s..

Stop retrying after a reasonable limit or allow the user to retry manually.

Bonus Challenges

(These are optional for participants who want to go further.)

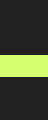
Bonus 1 : Typing Indicator

Bonus 2 : Authentication

Bonus 3 : Multiple Server Instances

Run two WebSocket server instances.

Client A



Server A

Client B



Server B

Use Redis Pub/Sub so that messages can travel between users connected to different servers.

Good Luck!!